

UNIT-2.

2.1 Software Design

- Design process & concepts
- Effective Modular Design
- Design Model.
- Design Documentation
- Approaches to Software Design

Software Design Process & Concepts

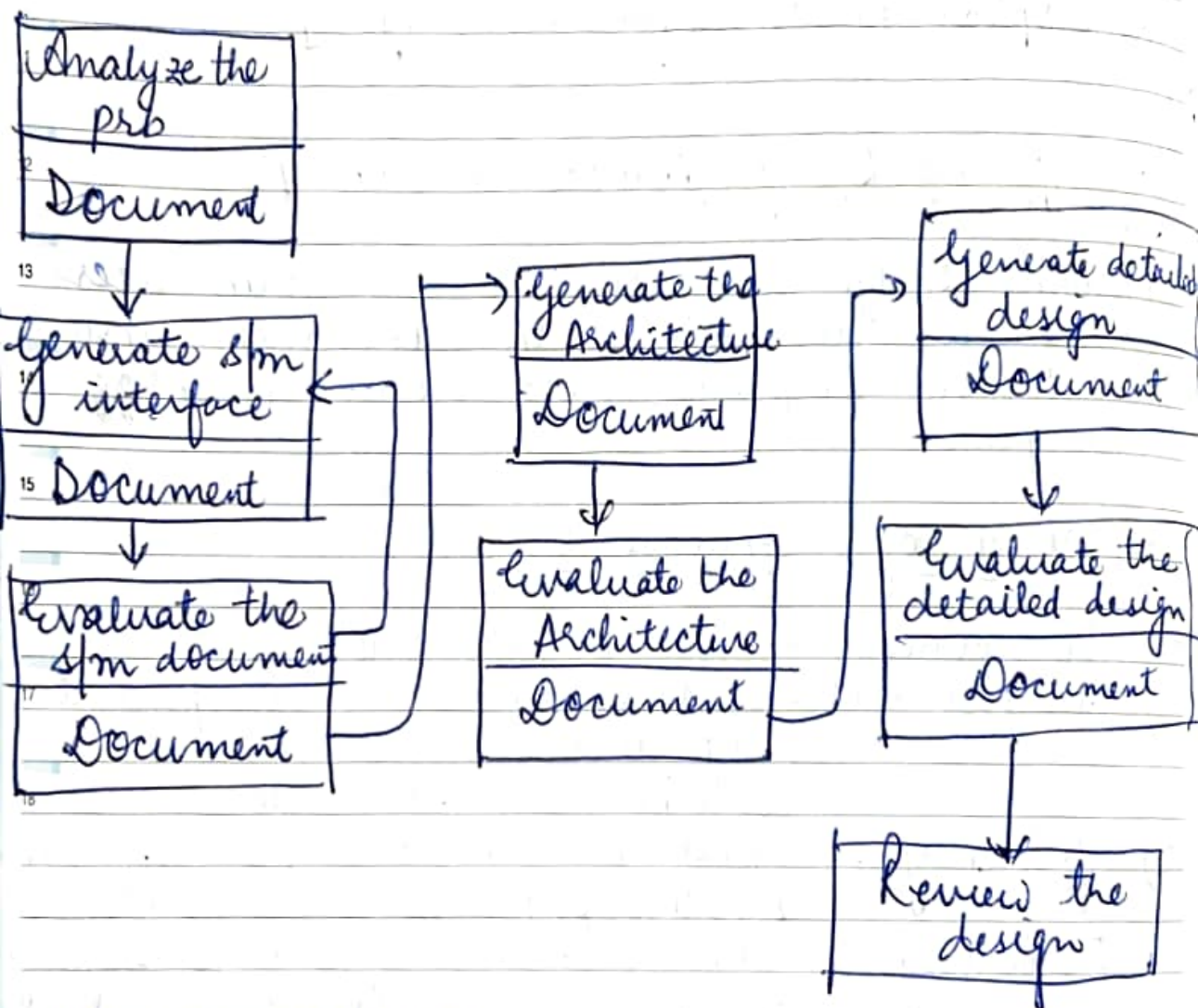
↓ Software Design is process to transform user req. as described in SRS documents into a form implementable using a prgg. language.

Objectives of Software Design:-

- 1) Correctness → as per req.
- 2) Completeness → should have all components like data structures, modules & external interfaces etc.
- 3) Efficiency → Resources should be used efficiently by the prg.
- 4) Flexibility → Able to modify on changing needs.
- 5) Consistency → no inconsistency in the design.
- 6) Maintainability → design should be so simple so that it can be easily maintainable by other designers.

Software Design Levels :- 3 levels!

- Interface Design
- Architectural Design
- Detailed Design



Interface Design

It is the specification of interaction b/w system & environment.

- P.T.O.

FEBRUARY
SUNDAY
WK 07 • 044-321

13

MTWTFSS
10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30

b) Architectural Design :-

- It is the specification of major components of s/m, their responsibilities, properties, interfaces & the relationships & interactions b/w them.
- In this, overall structure of s/m is chosen & the internal details of major components are ignored.

Architectural Design includes :-

- i) Gross decomposition of s/ms into major components.
- ii) Allocation of functional responsibilities to components.
- iii) Component interfaces.
- iv) ^{Component} Scaling & performance properties, resource consumption properties, reliability properties.
- v) Communication & interaction b/w components.

c) Detailed Design :-

- It is the specification of internal elements of all major s/m components, their properties, relationships, processing & other their algorithms.

MARCH 2022
 S M T W T F S S
 1 2 3 4 5 6 7 8 9 10 11 12 13
 14 15 16 17 18 19 20 21 22 23 24 25 26 27
 28 29 30 31

14

FEBRUARY
 MONDAY
 045-320 • WK 08

and data structures.

The detailed design includes :-

- i) Decomposition of major s/w components into program units.
- ii) Allocation of functional responsibilities to
- iii) User Interfaces.
- iv) Unit states & changes.
- v) Data & Control interaction b/w units.
- vi) Data packaging & implementation.
- vii) Algorithms & Data Structures.

Modularization

- It is a technique to divide a software system into multiple discrete & independent modules which are expected to be capable of carrying out tasks independently.
- Designers tend to design modules such way that they can be executed & compiled separately & independently.

FEBRUARY
TUESDAY
WK 08 • 046-319

15

JANUARY							FEBRUARY						
M	T	W	T	F	S	S	M	T	W	T	F	S	S
					1	2	3	4	5	6	7	8	9
10	11	12	13	14	15	16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31						

→ Modular Design follows the rule of 'divide & conquer'.

Advantages of modularization

- i) Smaller components are easier to maintain.
- ii) Program can be divided based on functional aspects.
- iii) Desired level of abstraction can be brought in the program.
- iv) Components with high cohesion can be re-used again.
- v) Concurrent execution can be made possible.
- ~~vi) Desired~~

Need of modular Design in SW

→ Designing a complete ^{sfm} in one go comprising of each & every required functionality is a hectic work, so we breakdown the complete sfw into various modules.

→ A module is defined as the unique & addressable components of software which can be solved & modified independently without disturbing other modules of the sfw.

MARCH 2022
S S M T W T F S S
1 2 3 4 5 6 7 8 9 10 11 12 13
14 15 16 17 18 19 20 21 22 23 24 25 26 27
28 29 30 31

In order to build a s/w with effective modular design, there is a factor Functional Independence which comes into play.

Functional Independence is that ^afunction is atomic in nature so that it performs only a single task of s/w without or with least interaction with other modules.

Larger functional independence, results in good design & better quality.

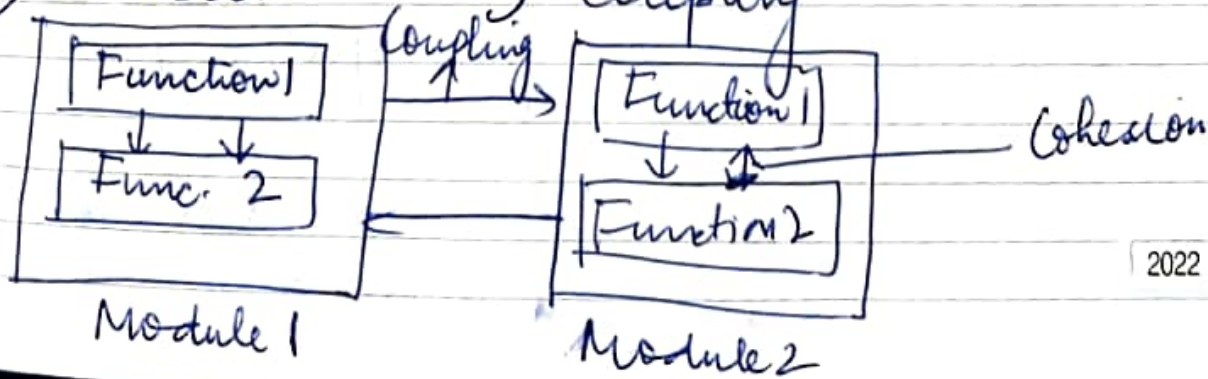
Benefits :- ① Making changes in a module is easy without affecting the whole s/w as modules are independent (no dependency) to each other.

② Developers gets a clear requirement of each & every functions & hence designing of s/w becomes easy & error-free.

Imp Independence of modules of software system can be measured using 2 criteria :-

1) Cohesion

2) Coupling



FEBRUARY
THURSDAY
WK 08 • 048-317

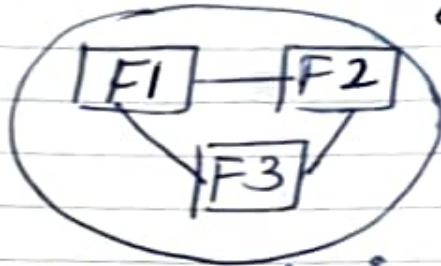
17

Both coupling & cohesion are important factors in SW system's design.

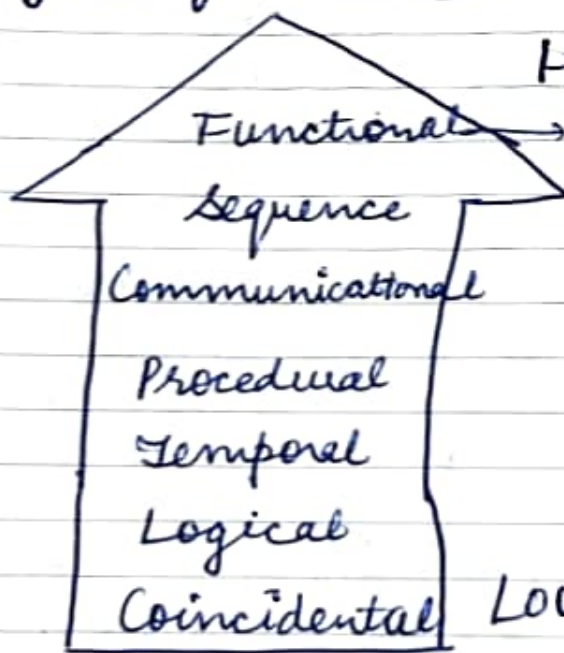
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

COHESION :- refers to the degree to which elements within a module work together to fulfil a single, well-defined purpose.

High cohesion means elements are closely related & focused on a single purpose & vice-versa.



Seven types of cohesion:-



High
strongest form of cohesion.

1) Functional Cohesion :- elements within a module are grouped based on single, specific functionality or task.

2) Sequential Cohesion :- elements are organized in a linear sequence where the output of one element



18

FEBRUARY
FRIDAY
049-316 • WK 08

becomes the input of the next.

- 3) Communicational → elements within a module work together to manipulate a shared data structure. They might not perform the same function, but their actions are closely related to a common piece of data.
- 4) Procedural → elements are grouped based on their involvement in a specific sequence of actions or steps.
- 5) Temporal → elements are grouped b'coz they need to be executed at the same time or during the same phase.
- 6) Logical → when logically categorized elements are put together into a module, it is called logical cohesion.
- 7) Co-incidental → elements are grouped arbitrarily without a clear, meaningful relationship. This type of cohesion typically indicate poor modular design.

P.T.O.

FEBRUARY
SATURDAY
WK 08 • 050-315

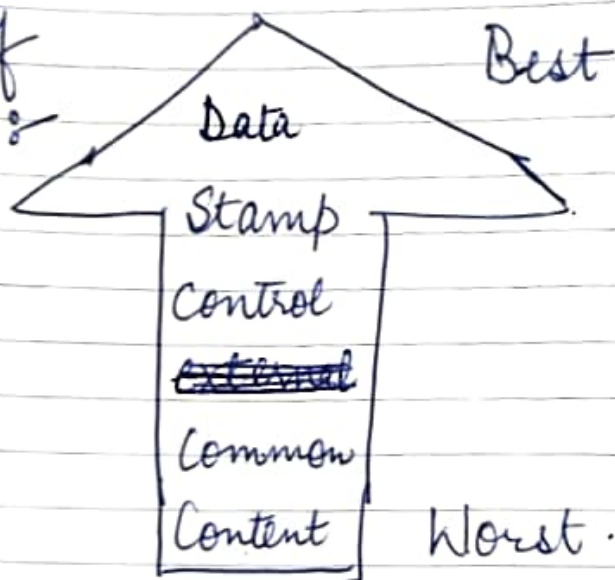
19

MTWTFSS
1 2 3 4 5 6
10 11 12 13 14 15 16
20 21 22 23 24 25 26
27 28 29 30

COUPLING:- It refers to the degree of interdependence b/w different modules, classes or components of a s/m software.

→ High coupling means strong interconnections where changes in one module can cascade through others, while low coupling means greater independence & isolation b/w modules.

5 types of coupling:-



1) Content Coupling → Modules share data directly through global variables or parameters. (strongest coupling & not recommended.) as modules are highly dependent on each other.

2) Common coupling → when ^{multiple} modules have read & write access to some global data.

MARCH 2022
M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8 9 10 11 12 13
14 15 16 17 18 19 20 21 22 23 24 25 26 27
28 29 30 31

3) Control Coupling → Two modules are control-coupled if one of them decides the function of other modules or changes its flow of execution.

4) Stamp Coupling → when multiple modules share common data structures work on different part of it.

5) Data Coupling → when 2 modules interact with each other by means of passing data (as parameter).

Ideally, no coupling is considered to be best.

A Good modular design should have High Cohesion and Low Coupling.

FEBRUARY
MONDAY
WK 09 • 052-313

21

MTWTFSSMTWTFSS
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

3' Design Model

↳ to describe a system's implementation & source code in a diagrammatic fashion.

Various design elements:-

- 1) Data Design elements → ^{model of data that represent high level}
- 2) Architectural " " → overall view of s/m
- 3) Interface " " → info flow within it & out of it
- 4) Component Level Diagram elements
- 5) Deployment " design elements.

↓
shows the functionality & subsystems that are allocated in the physical computing environment which support the

→ describes the internal details of each software component.

(How processing occurs in component)

4' Design Documentation:-

↳ collection of documents & resources that covers all aspects of product design.

↳ has info about users, product features, project deadline, implementation details and design decisions that your team & stakeholders have agreed on.

MARCH 2022
 M T W T F S S M T W T F S S
 1 2 3 4 5 6 7 8 9 10 11 12 13
 14 15 16 17 18 19 20 21 22 23 24 25 26 27
 28 29 30 31

22

FEBRUARY
 TUESDAY
 053-312 • WK 09

↳ Design documents are written by sw engineers / designers or project managers & further passed to sw development team to give them an overview of what needs to be built & how.

↳ A properly documented design should include the following info:-

- * Architecture design
- * Implementation plan
- * Installation guide.
- * Validation test plan
- * Operational procedures.

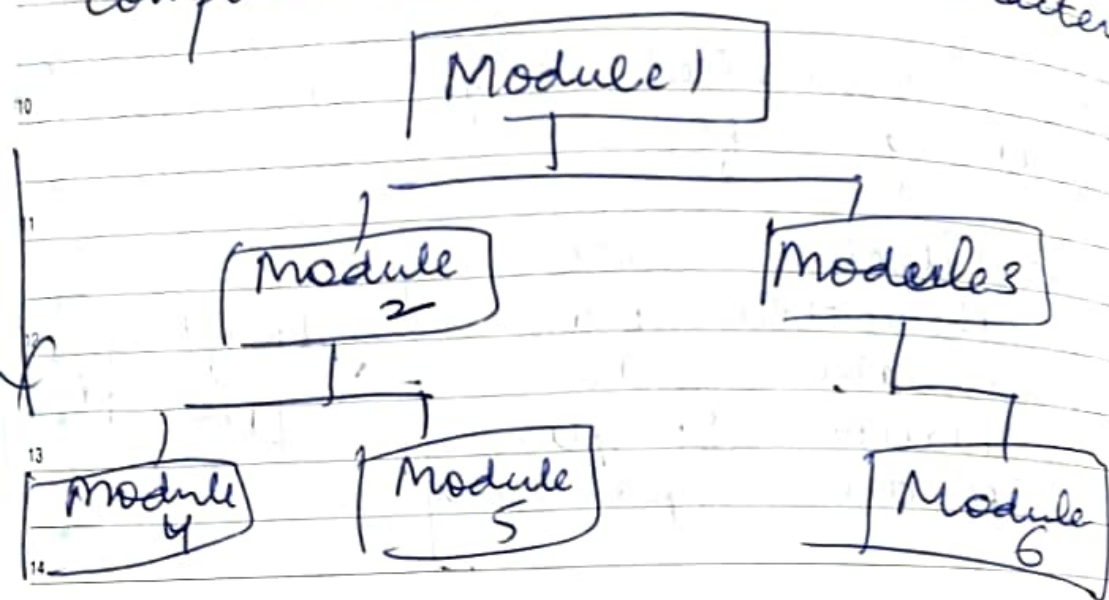
do from
LMS. ppt

Approaches of Sw Design:		
Structured Design	Function Oriented Design	Object-oriented Design
conceptualization of problem into several well organized elements of solution.	S/m is composed of many smaller sub-s/m's known as fns	oops & concepts
→ how problem is being solved (main focus)		

FEBRUARY
WEDNESDAY
WK 09 • 054-311

23

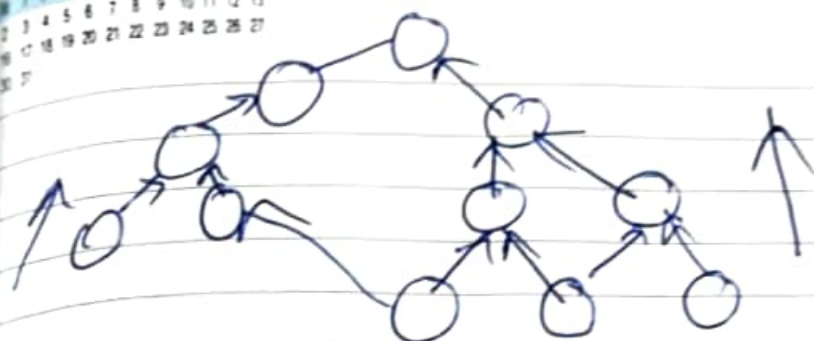
Top-down → takes the whole software as one entity & then decomposes to achieve more than one sub-spm or component based on some characteristics.



Bottom-up → starts with most specific & basic components, it proceeds with composing higher level of components by using basic or lower level components.

It keeps creating higher level components until the desired spm is not evolved as a single component. With each higher level, amt. of abstraction is red.

Both approaches are not practical. Instead a good combination of both is used.



Hybrid → combination of both